

TECHNICAL REPORT

ResNet50 Chest X-ray Classification

Author: Yasser Idbouzkri

2026

ABSTRACT

This term paper studies the problem of image classification using chest X-ray images and a pretrained convolutional neural network. The practical task is to distinguish between normal chest X-ray images and images showing pneumonia. The work is based on a clean PyTorch training pipeline that uses a reproducible procedure for data preparation, transfer learning, validation, threshold selection, and final testing. The theoretical part explains supervised learning, dense neural networks, convolutional neural networks, transfer learning, and the ResNet50 architecture. The practical part describes the dataset structure, preprocessing operations, ImageNet normalization, augmentation strategy, class imbalance handling, two-stage training, and model evaluation. After validation-based threshold adjustment, the final ResNet50 model used a decision threshold of 0.98 and achieved 90.87% accuracy, 95.99% ROC-AUC, 97.69% sensitivity for pneumonia, 79.49% specificity for normal images, and 89.88% macro F1 on the independent test set. The result demonstrates a strong educational classification pipeline while remaining limited to machine-learning experimentation and not clinical diagnosis.

Keywords: image classification, convolutional neural network, ResNet50, transfer learning, chest X-ray, pneumonia, PyTorch, validation, ROC-AUC, confusion matrix.

CONTENTS

ABSTRACT.....	2
INTRODUCTION.....	4
1 GENERAL APPROACHES TO IMAGE CLASSIFICATION.....	6
1.1 Description of supervised neural network training.....	6
1.2 Basic principles of dense neural networks.....	9
1.3 Basic principles of convolutional neural networks.....	11
1.4 Transfer learning and pretrained CNN models.....	14
1.5 ResNet50 architecture for image classification.....	17
1.6 Evaluation logic for medical image classification.....	19
1.7 Typical causes of failure in image classification projects.....	21
2 APPLICATION OF RESNET50 TO A CLASSIFICATION PROBLEM.....	24
2.1 Data preparation.....	24
2.2 Preprocessing and augmentation.....	26
2.3 Building the pretrained ResNet50 model.....	29
2.4 Training strategy and validation protocol.....	31
2.5 Testing the neural network model.....	34
2.6 Reproducibility and code organization.....	40
2.7 Practical interpretation of final results and limitations.....	42
CONCLUSION.....	45
THE LIST OF THE DOCUMENT SYMBOLS.....	47
REFERENCE LIST.....	48
APPENDIX A.....	49
APPENDIX B.....	51

INTRODUCTION

The relevance of the topic is determined by the rapid development of computer vision methods and their increasing use in practical image analysis tasks. Image classification is one of the central problems of computer vision because it transforms a visual object into a meaningful class label. In a medical context, this type of task is especially important because visual data can contain subtle patterns that are difficult to describe manually but can be learned by a neural network from a sufficiently organized dataset [1, 2, 3, 4, 5].

Chest X-ray analysis is a typical example of a classification problem in which images are visually similar, yet belong to clinically different categories. In this work, the two classes are normal chest X-ray images and chest X-ray images associated with pneumonia. The task is not treated as a medical diagnostic system; it is considered as an educational machine-learning problem that demonstrates how a convolutional neural network can be trained and evaluated on image data.

A reliable version of the project must be organized as a reproducible machine-learning pipeline. In a reliable computer vision project, the code must not only train a model; it must also define the dataset split, preprocessing operations, model architecture, optimization procedure, validation criterion, saved checkpoint, and final test evaluation. The new implementation therefore uses a clean PyTorch script instead of disconnected notebook cells.

The goal of the term paper is to solve the image classification problem by applying a pretrained ResNet50 convolutional neural network to a chest X-ray pneumonia dataset.

In order to achieve the set goal it is necessary to solve the following tasks: 1) to describe the theoretical principles of supervised neural network training; 2) to explain the role of dense neural networks and convolutional neural networks in image classification; 3) to describe the idea of transfer learning and the ResNet50 architecture; 4) to prepare the image dataset using resizing, normalization, augmentation, and stratified validation; 5) to build and train the pretrained ResNet50 model; 6) to evaluate the trained model using appropriate classification metrics; 7) to

present the training and testing procedure in a form that can be reproduced from the program code.

The object of the work is the image classification process. The subject of the work is the application of a pretrained convolutional neural network to the binary classification of chest X-ray images. The practical result of the work is a structured training pipeline that can be executed on a GPU environment such as Kaggle or Google Colab and then used to produce final plots and metrics for the report.

The work consists of an introduction, two main chapters, conclusion, the list of document symbols, reference list, and an appendix with selected program code fragments. The first chapter explains theoretical approaches to image classification. The second chapter presents the practical implementation of the ResNet50-based model and the evaluation protocol.

1 GENERAL APPROACHES TO IMAGE CLASSIFICATION

1.1 Description of supervised neural network training

Supervised neural network training is a learning process in which each input object is associated with a known target label. In image classification, the input object is an image represented as a tensor of pixel values, while the target label is a class name. The purpose of training is to learn a mapping from the input image space to the set of class labels, so that the model can make correct predictions for images that were not used during training [1].

The main elements of supervised training are the dataset, the model, the loss function, and the optimizer. The dataset provides examples. The model transforms each image into numerical outputs called logits. The loss function measures how far the model output is from the correct label. The optimizer changes the trainable parameters of the model in the direction that reduces the loss. This process is repeated for many batches and epochs until the model learns stable visual patterns.

A neural network does not receive semantic knowledge directly. It receives numbers. Therefore, every image must first be converted into a numerical tensor. In deep learning libraries such as PyTorch, the image is usually represented as a tensor with dimensions corresponding to color channels, height, and width. For pretrained ImageNet models, the input normally has three channels and is normalized using the mean and standard deviation of the original ImageNet training distribution [5, 6].

The training process is usually divided into several stages. First, the data are loaded and transformed. Second, the model computes predictions during the forward pass. Third, the loss function compares predictions with true labels. Fourth, the backward pass computes gradients. Fifth, the optimizer updates the parameters. This loop forms the basic mechanism of neural network learning. The relationship between these elements is shown in accordance with Figure 1.

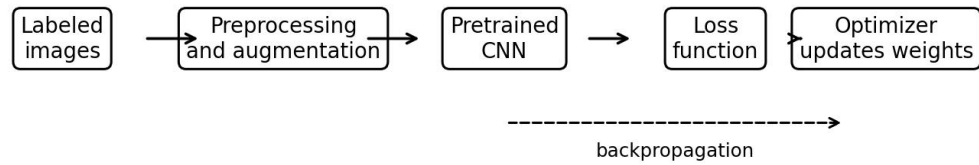


Figure 1 – Supervised training pipeline for image classification

Source: compiled by the author

A reliable training procedure must also include validation. The validation set is not used to update the weights; it is used to monitor how well the model generalizes. This distinction is essential because a model can obtain low training loss while failing on unseen data. Such a situation is called overfitting. The validation stage helps detect overfitting and supports informed model selection.

The test set has a different role. It should be used only after model selection is complete. In the present project, the test set is reserved for the final evaluation of the best saved model. This avoids the common error of repeatedly adjusting the model according to test results. If the test set influences training decisions, the reported performance becomes unreliable.

The quality of supervised training depends on several factors. The most important factors are the correctness of labels, the representativeness of images, the balance between classes, the choice of model, the preprocessing procedure, the learning rate, and the evaluation metrics. When these factors are not controlled, a neural network can produce apparently high accuracy while learning a weak or biased decision rule.

Medical image classification requires careful evaluation because the cost of different errors is not equal. For example, a model that predicts pneumonia for every image may obtain acceptable accuracy if pneumonia images are more frequent, but such a model would fail to recognize normal cases. For this reason, accuracy alone is

not sufficient in the present work. The report uses sensitivity, specificity, F1-score, and ROC-AUC in addition to accuracy.

Table 1 – Main components of supervised neural network training

Component	Function in the training process
Dataset	Provides labeled input examples used for learning, validation, and testing
Model	Transforms image tensors into class scores
Loss function	Measures the error between predictions and true labels
Optimizer	Updates trainable parameters according to gradients
Validation set	Estimates generalization during training without updating weights
Test set	Provides the final independent evaluation after model selection

Source: compiled by the author

The components listed in Table 1 are implemented explicitly in the new script. The script separates training, validation, and testing, applies different transformations for training and evaluation, and saves the best model according to validation performance. This organization is important because it turns a simple experiment into a reproducible machine-learning workflow.

A useful way to understand supervised learning is to separate the learning problem from the implementation details. The learning problem asks what relationship should be discovered between image and label. The implementation details define how images are loaded, how batches are formed, how gradients are computed, and how weights are saved. A well-written project should make both levels clear.

In this work, every training image contributes to the adjustment of model parameters only through the loss function. The model does not memorize file names or folder paths; it receives tensors and labels. This point is important because image classification must be evaluated on visual patterns, not on accidental properties of the dataset organization.

The batch is the practical unit of training. Instead of using one image at a time or the entire dataset at once, the DataLoader forms batches. The batch size affects memory consumption, speed, and gradient stability. A batch size of 32 is a reasonable compromise for common GPU environments and 224 x 224 images.

The epoch is the larger unit of training. One epoch means that the training loop has processed the training subset once. However, a higher number of epochs does not

automatically produce a better model. If the network starts to memorize training examples, validation performance may stop improving while training performance continues to improve.

For this reason, the script records both training loss and validation metrics. The comparison between them is more informative than a single final number. When both improve together, the model is likely learning useful patterns. When training improves but validation does not, overfitting becomes a serious possibility.

Another important concept is reproducibility. Machine-learning training contains randomness in data splitting, data augmentation, and parameter initialization of the new classifier head. The script therefore sets random seeds and stores split information. This does not make every run bit-identical on all hardware, but it makes the procedure controlled and auditable.

The training process should also be fail-safe. A script that silently proceeds with the wrong dataset path or the wrong class order can produce meaningless results. The new implementation checks for the expected folder structure and prints the detected class names and class counts before training begins.

1.2 Basic principles of dense neural networks

Dense neural networks, also called fully connected neural networks, are one of the most basic architectures in deep learning. In a dense layer, each neuron receives input from all neurons of the previous layer. The output of a neuron is computed from a weighted sum of inputs followed by an activation function. By stacking dense layers, the model can approximate complex non-linear relationships [2].

The advantage of dense neural networks is their generality. They can be used for classification, regression, and many other tasks. However, dense networks do not naturally exploit the spatial structure of images. If an image is flattened into a one-dimensional vector, information about neighboring pixels becomes less explicit. The network must then learn spatial relationships from a very large number of parameters.

For small tabular datasets, dense networks may be sufficient. For images, they are usually inefficient because they ignore locality and translation structure. A chest X-ray image contains local visual patterns such as edges, ribs, lung fields, and

regions of opacity. These patterns are better processed by convolutional filters than by fully connected layers alone.

In image classification pipelines, dense layers are still useful at the end of a convolutional network. After convolutional layers extract features, a classifier head can use dense layers or linear layers to transform the feature vector into class logits. In the ResNet50 pipeline, the final fully connected layer is replaced so that the pretrained network can output two classes instead of the original ImageNet classes.

Table 2 – Dense neural networks model usage advantages and disadvantages

Advantages	Disadvantages
Flexible architecture adaptable to many data types	Large number of parameters when used directly on images
Ability to model non-linear relationships	Weak use of spatial structure compared with convolutional networks
Simple theoretical interpretation	Higher risk of overfitting with limited image data
Useful as classifier layers after feature extraction	Requires careful regularization and sufficient training data
Can approximate complex functions	Computationally inefficient for high-resolution images

Source: compiled by the author based on [1, 2]

The comparison in Table 2 explains why dense networks are presented as a theoretical foundation but not chosen as the main model for the practical part. The chest X-ray classification problem requires extraction of visual patterns from two-dimensional image structures. Therefore, a convolutional neural network is more appropriate.

Another limitation of a purely dense model is that it would require many more parameters if the input image is large. For example, a 224 x 224 image with three channels contains more than 150 thousand input values. Connecting all these values directly to a dense hidden layer would create a large parameter matrix before any high-level visual representation is learned. This is one of the reasons why modern image classifiers use convolutional backbones before the final classifier.

Dense neural networks remain important even when they are not selected as the main architecture. They introduce the basic idea that a model can learn by combining weighted inputs, applying non-linear transformations, and adjusting parameters through gradient descent. The final classifier of ResNet50 is also a dense linear mapping from learned features to output classes.

In the context of this project, dense layers appear after the convolutional feature extractor. At that point, the image is no longer represented as raw pixels. It has already been transformed into a compact feature vector. A linear classifier can then separate the two classes in this learned representation space.

The limitation of dense networks is not that they are weak in general. Their limitation is that they do not include the correct inductive bias for images. An inductive bias is a structural assumption that helps the model learn efficiently. For images, locality and spatial hierarchy are important assumptions. CNNs include these assumptions directly.

If a dense network were trained on flattened X-ray pixels, it would treat two neighboring pixels and two distant pixels in a similar mathematical manner. This is inefficient because neighboring pixels usually form meaningful local patterns. Convolutional layers solve this by processing local neighborhoods with shared filters.

Dense networks also have a higher risk of overfitting when the input dimension is large. Many parameters can fit training noise rather than meaningful patterns. In a medical image dataset, this would be dangerous because the model might learn dataset-specific artifacts rather than visual signs related to the target classes.

For these reasons, the theoretical explanation of dense neural networks is included as a foundation, but the practical solution uses a convolutional pretrained backbone. This choice follows the nature of the input data and the goal of achieving a stable image classifier.

1.3 Basic principles of convolutional neural networks

Convolutional neural networks are a specialized class of neural networks designed for data with grid-like structure, especially images. A CNN processes an image using convolutional filters that slide across the spatial dimensions of the input. Each filter learns to detect a certain pattern, such as an edge, texture, curve, or more complex visual feature [1].

The key advantage of a convolution operation is parameter sharing. The same filter is applied at many spatial positions. This makes CNNs more efficient than fully connected networks for image data. It also helps the model recognize similar patterns

in different parts of the image. This property is useful for chest X-ray images because relevant visual signs may appear in different regions of the lungs.

A typical CNN contains convolutional layers, activation functions, normalization layers, pooling or downsampling operations, and final classification layers. Early layers usually learn low-level features such as edges and brightness changes. Middle layers learn more complex combinations of local features. Deeper layers can represent high-level patterns that are more directly related to the target classes. This hierarchy is shown in accordance with Figure 2.

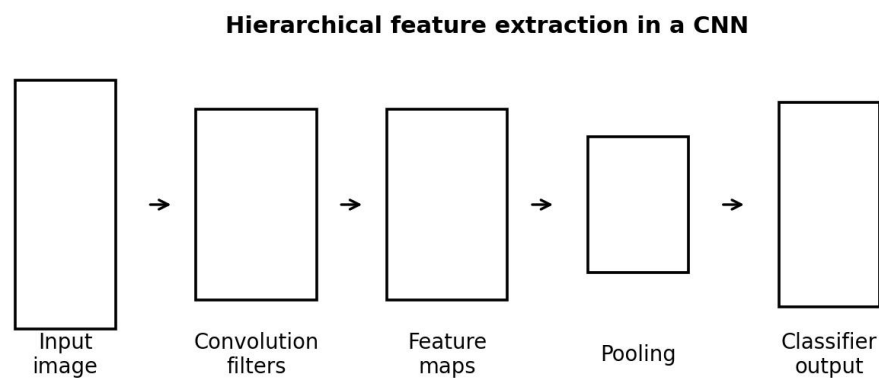


Figure 2 – Hierarchical feature extraction in a convolutional neural network

Source: compiled by the author

In medical image classification, the hierarchical nature of CNNs is valuable because the class decision often depends on a combination of local and global patterns. A model must recognize not only isolated pixel intensity changes but also their anatomical context. For pneumonia classification, the network may learn patterns related to opacity distribution, lung texture, and contrast between normal and abnormal regions.

Pooling and downsampling reduce spatial resolution while increasing the receptive field of deeper layers. This enables the network to process larger visual contexts. However, aggressive downsampling may also remove fine details. For this reason, modern CNN architectures are carefully designed to preserve enough information while controlling computational cost.

CNNs are powerful, but they still require a disciplined training procedure. Data augmentation, normalization, learning rate selection, class imbalance handling, and

independent testing remain important. A strong architecture cannot compensate for a broken experimental protocol. In the new project implementation, the architecture is combined with a structured pipeline in order to improve reliability.

Table 3 – CNN model usage advantages and disadvantages

Advantages	Disadvantages
Automatic extraction of local image features	Requires sufficient data and careful validation
Parameter sharing reduces computational cost	May overfit if the training set is small or biased
Captures spatial and hierarchical patterns	Architecture and hyperparameters affect performance
Works well for computer vision tasks	Interpretation is more difficult than in simple models
Can use pretrained weights through transfer learning	Preprocessing must match the pretrained model requirements

Source: compiled by the author based on [1, 2, 3]

Table 3 summarizes why CNNs are appropriate for the practical part of the work. The advantages directly match the needs of image classification, while the disadvantages show why the training script must include normalization, augmentation, validation, and metrics beyond accuracy.

A convolutional filter can be interpreted as a small pattern detector. During training, the values of the filter are adjusted so that the filter responds strongly to useful visual structures. In early layers, these structures may be very general. In deeper layers, combinations of earlier features become more task-specific.

The same filter is applied across the whole image. This means that the network can detect a similar local pattern regardless of where it appears. For a chest X-ray classification task, this is helpful because relevant visual information is not always in exactly the same position across patients and image acquisitions.

CNNs also reduce the number of parameters compared with dense networks. A small convolutional filter has the same parameters wherever it is applied. This allows the model to learn local patterns without creating a separate parameter for every pixel-to-neuron connection.

Normalization layers such as batch normalization help stabilize deep CNN training. They reduce internal distribution shifts and allow deeper models to be trained more effectively. ResNet50 contains normalization operations inside its residual blocks, which contributes to its practical stability.

Activation functions introduce non-linearity. Without non-linear activation functions, a deep network would collapse mathematically into a linear transformation. ReLU activation is commonly used because it is simple and computationally efficient.

The final layers of a CNN combine spatial features into a global representation. This representation is then used for class prediction. In ResNet50, global average pooling summarizes feature maps before the final classifier. This reduces the number of parameters and makes the classifier less dependent on exact spatial positions.

The strength of CNNs does not remove the need for careful data preparation. If images are resized incorrectly, normalized inconsistently, or evaluated with random augmentation, the model can become unstable. Therefore, architecture and preprocessing must be designed together.

In this project, the CNN is not trained from scratch. The pretrained backbone already contains learned convolutional filters. The practical task is to adapt these filters and the classifier to the binary chest X-ray problem through transfer learning.

1.4 Transfer learning and pretrained CNN models

Transfer learning is a technique in which a model trained on one dataset is adapted to another task. In computer vision, a common approach is to start from a CNN pretrained on a large dataset such as ImageNet and then replace or fine-tune its classifier for the target problem [4]. This approach is useful when the target dataset is much smaller than the dataset used for pretraining.

A pretrained CNN has already learned general visual features. Early filters can detect edges, gradients, textures, and basic shapes. Although chest X-ray images are different from everyday ImageNet images, many low-level visual operations remain useful. Transfer learning can therefore provide a better starting point than random initialization.

There are several transfer learning strategies. The simplest strategy is feature extraction: the pretrained backbone is frozen and only the final classifier is trained. A more flexible strategy is fine-tuning: some deeper layers of the pretrained model are unfrozen and trained with a smaller learning rate. The new script uses both strategies

in sequence. First, it trains the classifier head. Then, it fine-tunes the last feature block and the classifier.

This two-stage approach reduces the risk of damaging pretrained representations at the beginning of training. If all weights are trained immediately with a large learning rate, the network may forget useful features. By freezing the backbone first, the classifier learns to map pretrained features to the two target classes. After that, careful fine-tuning adapts the last layers to the chest X-ray domain.

Table 4 – Transfer learning strategies for CNN image classification

Strategy	Description	Use in the project
Feature extraction	Freeze pretrained convolutional backbone and train only the classifier	Used in the first training stage
Partial fine-tuning	Unfreeze only the last feature block and classifier	Used in the second training stage
Full fine-tuning	Train the entire network with a small learning rate	Not used by default to avoid overfitting
Training from scratch	Initialize all weights randomly	Not selected because the dataset is limited

Source: compiled by the author

The choice of pretrained ResNet50 is a balance between model strength, stability, and explainability. ResNet50 is not the newest or largest CNN architecture, but it is widely used, well documented, and easier to defend academically than a more obscure architecture. It is also supported directly by TorchVision, which simplifies reproducibility [5].

Transfer learning also imposes strict preprocessing requirements. The input images should be resized to the expected resolution and normalized using the mean and standard deviation associated with ImageNet pretrained weights. If this step is ignored, the model receives inputs with a distribution different from the one used during pretraining. This can significantly reduce performance and make training unstable.

In the practical script, ImageNet normalization is applied after conversion to tensor. Training images also receive augmentation, while validation and test images use only deterministic preprocessing. This separation prevents artificial transformations from affecting the final evaluation and keeps the test set representative of unseen data.

Transfer learning is especially useful in student projects because it reduces training time and improves the probability of obtaining meaningful results. Training a deep CNN from scratch usually requires a very large dataset and substantial computational resources. A pretrained model allows the project to focus on correct adaptation and evaluation.

The first stage of transfer learning in the script is intentionally simple. Only the classifier head is trained. This stage can be understood as asking whether the pretrained features are already sufficient to separate the two classes. If the validation metrics improve, it means that the generic visual representation contains useful information for the target task.

The second stage allows limited adaptation. By unfreezing the last residual block, the model can modify higher-level features that are closer to the classification decision. Lower layers remain frozen by default because they contain general visual filters that are less likely to require strong changes.

The learning rate difference between the two stages is important. The classifier head starts from random initialization, so it can tolerate a larger learning rate. The pretrained layers already contain useful information, so they should be fine-tuned with a smaller learning rate to avoid destructive updates.

Transfer learning also improves interpretability at the project level. It is easier to explain why the model begins with known pretrained weights than to justify a random deep network trained from zero on a limited dataset. This makes the method suitable for a term paper where both implementation and explanation are required.

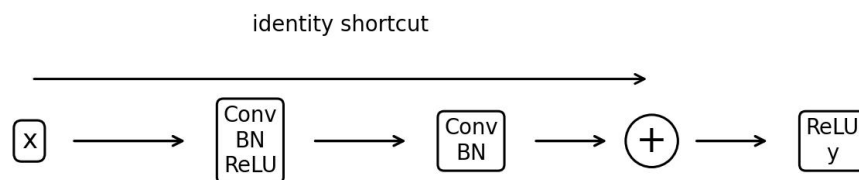
A possible limitation is domain mismatch. ImageNet images are natural images, while chest X-ray images are medical grayscale images. The transfer is not perfect. However, pretrained low-level filters and stable architecture still provide a useful starting point, especially when combined with careful fine-tuning.

The report therefore presents ResNet50 not as the only possible model, but as a strong and disciplined baseline. More advanced backbones could be added later, but the immediate priority is a correct experimental protocol.

1.5 ResNet50 architecture for image classification

ResNet50 is a convolutional neural network architecture based on residual learning. It was introduced to address the degradation problem that appears when very deep neural networks become difficult to optimize. Instead of forcing each block to learn a complete transformation, a residual block learns a correction that is added to the input through a shortcut connection [3].

The central idea of ResNet is the identity shortcut. If the optimal transformation is close to the identity mapping, the block can preserve the input signal and learn only a residual modification. This makes it easier to train deeper networks because gradients can flow through shortcut paths. The simplified structure of a residual block is presented in accordance with Figure 3.



A residual block learns a correction while preserving the input signal

Figure 3 – Simplified residual block used in ResNet architecture

Source: compiled by the author

The ResNet50 architecture contains an initial convolutional stem followed by four main residual stages. As the data pass through these stages, the spatial resolution decreases while the number of feature channels increases. This allows the network to move from low-level image features to more abstract representations.

In the original ImageNet configuration, ResNet50 ends with a fully connected layer that predicts 1000 classes. For the current binary classification task, this layer is replaced by a new classifier head that outputs two logits: one for the normal class and one for the pneumonia class. This replacement is necessary because the target label space is different from the pretraining label space.

The practical implementation also includes dropout before the final linear layer. Dropout is a regularization method that randomly disables part of the representation

during training. It can reduce overfitting by discouraging the classifier from depending too strongly on a small set of features. In the script, dropout is applied only in the classifier head.

ResNet50 is appropriate for this work because it is strong enough to serve as a serious pretrained baseline, while still being understandable and feasible to train on common GPU platforms. It provides a clear architecture for academic explanation and a stable foundation for the new reproducible pipeline.

Table 5 – Main ResNet50 components used in the project

Component	Purpose
Convolutional stem	Initial extraction of low-level visual features
Residual stage 1	Early residual feature learning
Residual stage 2	Intermediate feature abstraction and downsampling
Residual stage 3	Deeper representation of image patterns
Residual stage 4	High-level visual features used for classification
Classifier head	Mapping of extracted features to NORMAL and PNEUMONIA classes

Source: compiled by the author

The architecture described in Table 5 is not implemented manually from zero. Instead, the script uses the official TorchVision implementation of pretrained ResNet50 and modifies the final classification layer. This approach reduces implementation risk and improves reproducibility.

The residual connection is not merely a technical detail. It changes the optimization landscape of deep networks. Instead of learning a completely new transformation at each block, the network can learn a residual adjustment. This makes very deep networks easier to optimize and explains why ResNet architectures became a standard baseline in computer vision.

The number 50 in ResNet50 refers to the depth of the architecture in terms of weighted layers. This depth allows the network to learn complex visual abstractions. At the same time, the residual design prevents the training process from becoming as unstable as it would be in a plain deep network without shortcuts.

In the pretrained model, the earlier layers respond to generic visual signals. These signals are not specific to pneumonia but can still be useful for processing image structure. Deeper layers become more semantic in the original pretraining task. Fine-tuning the last block allows these higher-level representations to become more suitable for X-ray classification.

Replacing the final layer is mandatory because the output space changes. The original network was trained to distinguish many everyday object categories. The present model needs only two outputs. This replacement is a standard transfer learning operation and is implemented directly in the model-building function.

Dropout in the classifier head is used conservatively. It does not change the pretrained backbone. It only regularizes the newly trained classifier. This is appropriate because the classifier has to learn from the target dataset and may overfit if it becomes too confident too early.

The use of a standard architecture also helps with debugging. If training fails, the problem is less likely to be caused by an unknown custom architecture and more likely to be found in data loading, preprocessing, learning rate, class imbalance, or evaluation. This is useful for an educational project.

For these reasons, ResNet50 is a defensible choice: it is strong, documented, reproducible, and understandable. The goal is not to claim that it is the strongest possible CNN, but to build a reliable classification pipeline around a well-known pretrained model.

1.6 Evaluation logic for medical image classification

Evaluation is one of the most important parts of a classification project. A model can only be considered successful if it performs well on unseen data and if its performance is measured by metrics appropriate to the task. For a binary chest X-ray classification problem, it is not enough to report accuracy alone.

Accuracy measures the proportion of correct predictions among all predictions. It is useful but can be misleading when classes are imbalanced. If pneumonia images are more frequent than normal images, a model can obtain moderate accuracy by predicting pneumonia too often. Therefore, the project also reports precision, recall, specificity, F1-score, macro F1, and ROC-AUC.

Sensitivity, also called recall for the positive class, measures how many pneumonia cases are correctly identified. Specificity measures how many normal cases are correctly identified. Both are necessary because the two types of error have

different meanings. False negatives correspond to pneumonia images predicted as normal, while false positives correspond to normal images predicted as pneumonia.

The confusion matrix is especially important because it shows the exact number of true normal, false pneumonia, false normal, and true pneumonia predictions. Unlike a single scalar metric, the confusion matrix reveals whether the model collapses toward one class. The new script always saves and plots the test confusion matrix.

ROC-AUC evaluates the ranking quality of predicted probabilities independently from a fixed threshold. The threshold itself is selected on the validation set. In the script, the default threshold mode uses Youden's criterion, which balances sensitivity and specificity. The implementation of these metrics can be supported by standard machine-learning tools such as scikit-learn [7]. The selected threshold is then applied once to the test set.

Table 6 – Evaluation metrics used in the classification pipeline

Metric	Meaning in this project
Accuracy	Overall proportion of correct predictions
Precision for pneumonia	Proportion of predicted pneumonia cases that are correct
Sensitivity for pneumonia	Proportion of true pneumonia cases detected by the model
Specificity for normal	Proportion of true normal cases recognized as normal
F1-score	Harmonic balance between precision and recall
Macro F1	Average F1-score across both classes
ROC-AUC	Quality of probability ranking across thresholds
Confusion matrix	Exact distribution of correct and incorrect predictions

Source: compiled by the author

The metrics in Table 6 are included in the new script output. This is an improvement over an evaluation based only on loss or accuracy curves. In a medical image classification task, the final report should present not only whether the model is accurate, but also how the errors are distributed between the two classes.

The confusion matrix contains four values: true negatives, false positives, false negatives, and true positives. In this project, NORMAL is treated as the negative class and PNEUMONIA as the positive class. This convention makes it possible to compute sensitivity and specificity in a clear way.

Precision and recall answer different questions. Precision asks how many predicted pneumonia cases are actually pneumonia. Recall asks how many true

pneumonia cases were found by the model. Both are useful because a model can have high recall but low precision, or high precision but low recall.

Macro F1 is included because it gives equal importance to both classes. This is important when the dataset is imbalanced. A majority-class model may obtain acceptable accuracy but poor macro F1 because the minority class is not handled properly.

ROC-AUC is threshold-independent. It evaluates whether pneumonia images generally receive higher predicted probabilities than normal images. This is useful before fixing a classification threshold, but it does not replace the confusion matrix because practical predictions still require a threshold.

The threshold is selected using validation data, not test data. This preserves the independence of the test set. If the threshold were adjusted after looking at test results, the reported test metrics would become optimistic and less reliable.

The evaluation logic in the script is therefore designed to prevent the most common error in imbalanced classification: reporting a single high accuracy while hiding weak performance on one class.

1.7 Typical causes of failure in image classification projects

A technically strong neural network can still fail if the experimental protocol is weak. In student image classification projects, the most common problem is not the lack of a powerful model, but the lack of a controlled pipeline. A model trained in disconnected notebook cells can easily use inconsistent transformations, repeated training states, or incomplete evaluation.

One frequent failure is evaluation by accuracy only. Accuracy is simple to understand, but it hides class-level behavior. In an imbalanced dataset, a model may obtain acceptable accuracy by predicting the majority class too often. Such a model is not useful because it does not learn a balanced decision boundary.

Another failure is using the test set during model selection. If the student repeatedly checks test metrics and changes hyperparameters according to them, the test set is no longer independent. The final result then becomes overly optimistic. The

new script avoids this by using validation data for selection and reserving the test data for the end.

A third failure is missing normalization for pretrained models. Pretrained CNNs are not generic functions that accept any numerical scale. They were trained with a specific preprocessing convention. If this convention is ignored, the early layers receive shifted input values and may behave unpredictably.

A fourth failure is uncontrolled class imbalance. If one class dominates the training set, the loss function may reward the model for favoring that class. Class-weighted loss and balanced metrics are therefore not optional details; they are part of the scientific validity of the experiment.

A fifth failure is training the entire pretrained model too aggressively. A high learning rate applied to all layers can destroy useful pretrained representations. The two-stage strategy in this project is designed specifically to avoid this problem.

A sixth failure is not saving the best model. If only the final epoch is saved, the model used for testing may not be the model with the best validation behavior. The script therefore saves the best checkpoint whenever validation performance improves.

A seventh failure is poor documentation of code and outputs. If the training settings, class counts, selected threshold, and output files are not saved, the report becomes difficult to verify. The new pipeline writes these artifacts automatically.

These failure modes show why a structured pipeline is necessary. The goal was not simply to replace one architecture by another. The main goal was to replace an unreliable experimental style by a complete and defensible workflow.

The checklist in Table 7 summarizes the main risks and how the new implementation addresses them.

Table 7 – Failure modes and safeguards in the rebuilt pipeline

Failure mode	Safeguard in the new script
Accuracy-only evaluation	Reports sensitivity, specificity, macro F1, ROC-AUC and confusion matrix
Test leakage	Uses validation for selection and test only for final evaluation
Missing pretrained normalization	Applies ImageNet mean and standard deviation
Class imbalance	Computes class weights from the training subset
Aggressive fine-tuning	Uses frozen head training before limited fine-tuning
Saving only final epoch	Saves best validation checkpoint
Unclear outputs	Stores metrics, plots, predictions and split information

Source: compiled by the author

2 APPLICATION OF PRETRAINED RESNET50 TO A CLASSIFICATION PROBLEM

2.1 Data preparation

The practical part of the work applies a pretrained ResNet50 model to a chest X-ray image classification problem. The dataset is organized into two classes: NORMAL and PNEUMONIA. The input data are image files stored in class-specific folders, which allows the ImageFolder utility from TorchVision to assign labels automatically according to folder names [5]. The dataset formulation follows the public chest X-ray pneumonia classification resource and related medical imaging work [8, 9].

The dataset structure expected by the script contains separate train and test folders. The training folder includes the images used for learning, while the test folder is reserved for final evaluation. A validation set is created by a stratified split from the training folder. This choice is used because some versions of the chest X-ray dataset contain a very small validation folder, which is not sufficient for stable model selection.

The use of a stratified validation split ensures that the class proportions in the validation set remain similar to the class proportions in the training set. This is important because the dataset is imbalanced. Pneumonia images are more frequent than normal images. If the validation split were random without stratification, the minority class could be underrepresented, making validation metrics unstable.

Examples of the image classes are presented in accordance with Figure 4. The figure illustrates the two target classes used in the binary classification task: NORMAL and PNEUMONIA. The important requirement is that both classes are represented clearly and that the figure source is indicated in the report.

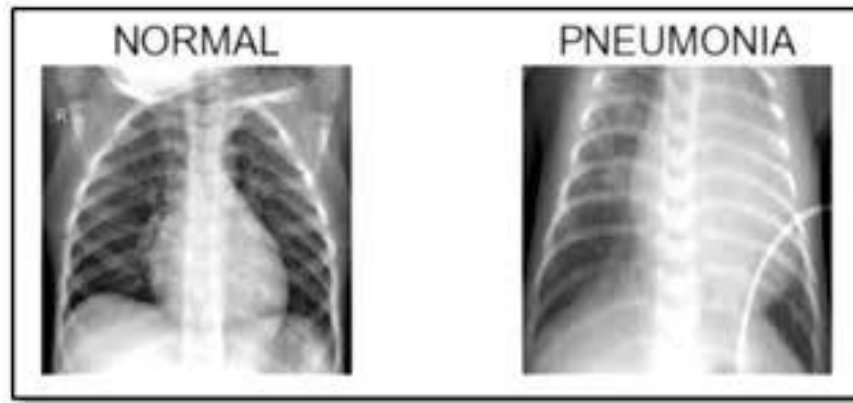


Figure 4 – Examples of chest X-ray images from two classes

Source: compiled by the author

The dataset contains a total of 5,863 labeled images in the common version used for this educational task. The approximate class distribution is shown in Table 8. The number of pneumonia images is significantly larger than the number of normal images, which motivates the use of class-weighted loss during training.

Table 8 – Distribution of chest X-ray images in the dataset

Class	Approximate number of images	Role in classification
NORMAL	approximately 1,500	Negative class
PNEUMONIA	approximately 4,000	Positive class
Total	approximately 5,863	Complete labeled dataset

Source: compiled by the author

The class distribution shown in Table 8 has direct consequences for training and evaluation. If class imbalance is ignored, the model may learn to favor the majority class. The new script therefore computes class weights from the actual training subset and passes them to the cross-entropy loss function. This encourages the optimizer to pay more attention to the minority class during learning.

The dataset root is detected automatically when possible. The script expects a folder containing train/NORMAL, train/PNEUMONIA, test/NORMAL, and test/PNEUMONIA. If the root path is not provided, the script attempts to download or locate the dataset using KaggleHub. If this fails, it raises an explicit error explaining how to pass the dataset path manually.

The ImageFolder organization is convenient because it connects the physical folder structure with class labels. However, it also requires discipline. The folder names must be checked carefully because the alphabetical order determines

numerical label indices. The script prints `class_to_idx` so that the student can verify which class is assigned to 0 and which class is assigned to 1.

The positive class in this report is PNEUMONIA. The probability extracted from the softmax output corresponds to the pneumonia class if the class order is NORMAL followed by PNEUMONIA. If a different dataset organization changes the class order, the evaluation logic must be checked before interpreting metrics.

The test folder is treated as independent. It should not be used to decide the learning rate, epoch number, augmentation policy, or threshold. This is a methodological rule, not a programming detail. The value of a test set comes from the fact that it simulates new unseen data.

The validation split from the training folder is created with a fixed random seed. This improves comparability between runs. If the split changed randomly every time, the validation curve could change simply because the validation images changed, not because the model improved.

Class imbalance is visible already at the dataset preparation stage. The majority class can dominate the loss if all errors are treated equally. The class-weighted loss gives a larger weight to the minority class so that normal images contribute more strongly to the optimization objective.

The split information is saved to a JSON file. This file documents how many images were used for training, validation, and testing, and how many images of each class appeared in each subset. This information should be preserved because it supports the credibility of the final report.

2.2 Preprocessing and augmentation

Preprocessing is necessary because neural networks require numerical tensors with consistent dimensions. The ResNet50 pretrained model expects images of a fixed size and normalized input values. In the implemented pipeline, all images are resized to 224 x 224 pixels. This size is compatible with common ImageNet pretrained models and reduces computational cost compared with larger resolutions.

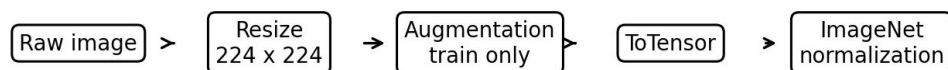
After resizing, each image is converted to a tensor. Although X-ray images are often grayscale, ImageFolder loads them in a format compatible with three-channel

models. This is important because the pretrained ResNet50 model was trained on three-channel natural images. The model architecture therefore expects three input channels.

The next step is ImageNet normalization. This operation subtracts the ImageNet mean and divides by the ImageNet standard deviation for each channel. This does not make the X-ray images identical to natural images, but it makes the input numerical scale compatible with the pretrained weights. Without this step, the pretrained filters receive inputs outside their expected distribution.

Data augmentation is applied only to the training set. The purpose of augmentation is to increase robustness by exposing the model to small variations of the same image. The script uses slight rotation, small affine transformations, scale variation, translation, and horizontal flipping. These transformations are moderate because medical images should not be distorted aggressively.

The preprocessing and augmentation pipeline is summarized in accordance with Figure 5. The distinction between training transformations and evaluation transformations is essential: validation and test images must not receive random augmentation, because evaluation must be deterministic and comparable across runs.



The same evaluation transform is used for validation and testing

Figure 5 – Data preprocessing and augmentation pipeline

Source: compiled by the author

Table 9 – Image transformations used in the ResNet50 pipeline

Stage	Transformation	Purpose
Training	Resize to 224 x 224	Standardize input size
Training	Random rotation up to a small angle	Improve robustness to minor positioning differences
Training	Small affine transformation	Simulate slight translation and scale changes
Training	Horizontal flip	Increase training diversity

Training and evaluation	Conversion to tensor	Represent image as numerical data
Training and evaluation	ImageNet normalization	Match pretrained model input scale
Validation and test	Deterministic resize only	Keep evaluation stable and reproducible

Source: compiled by the author

Table 9 shows that the preprocessing stage is not a cosmetic part of the project. It is a necessary interface between image files and the pretrained neural network. The old style of experimentation often fails because transformations are added informally or repeated in different notebook cells. The new script defines transformations in one function, which makes the pipeline easier to inspect and reproduce.

The augmentation choices are intentionally conservative. In chest X-ray classification, large rotations or strong color changes would be inappropriate because they may create unrealistic images. The goal is not to generate artificial medical cases, but to make the model slightly less sensitive to harmless variations in image acquisition and positioning.

Another important detail is that augmentation is applied after the train-validation split. The validation subset uses the evaluation transform even though it originates from the training folder. This prevents random augmentation from changing validation metrics at every epoch. As a result, validation curves become more meaningful.

The choice of image size is a compromise. A larger image may preserve more detail, but it increases memory use and training time. A smaller image is faster but may remove useful patterns. The size 224 x 224 is chosen because it matches the standard input size of many ImageNet pretrained models and works reliably with common GPU resources.

Normalization is sometimes misunderstood as a minor operation. In transfer learning, it is essential. The pretrained weights were optimized under a specific input distribution. Feeding unnormalized images to a pretrained model can shift activations and reduce the usefulness of learned filters.

Augmentation should respect the semantics of the data. In natural image classification, strong random crops or color jitter may be useful. In chest X-ray

classification, such transformations could distort medically meaningful structures. Therefore, the augmentation policy is intentionally mild.

Horizontal flipping is included because it can increase variation, but it should be used with awareness. For this educational dataset, the task is a binary normal-pneumonia classification rather than localization of anatomical laterality. If the task were more clinically specific, augmentation would require additional domain review.

The validation and test transforms exclude random operations. This ensures that the same image always produces the same input tensor during evaluation. Deterministic evaluation is necessary for stable metrics, reproducible plots, and fair comparison between model versions.

The transformation function is centralized in the script. Centralization prevents the common notebook problem where different cells use slightly different preprocessing. A single definition makes the experiment easier to audit and less likely to contain hidden inconsistencies.

2.3 Building the pretrained ResNet50 model

The model used in the practical part is ResNet50 pretrained on ImageNet. The implementation is loaded from TorchVision. The final fully connected layer of the original network is replaced because the original model predicts 1000 ImageNet classes, while the present task requires two classes only: NORMAL and PNEUMONIA [5].

The modified classifier head consists of a dropout layer followed by a linear layer. Dropout provides regularization, while the linear layer maps the extracted feature vector to two output logits. The model then uses the softmax operation internally during evaluation to obtain the probability of the pneumonia class.

The script supports several architectures, but the default and recommended model is ResNet50. ResNet18 is included for quick testing, DenseNet121 is included as a strong medical-imaging alternative, and EfficientNet-B0 is included for comparison with the earlier report. However, the present report focuses on ResNet50 to keep the methodology coherent.

Table 10 – Model configuration used in the project

Element	Selected value
Backbone	ResNet50 pretrained on ImageNet
Input resolution	224 x 224 pixels
Number of output classes	2
Class names	NORMAL, PNEUMONIA
Classifier regularization	Dropout before final linear layer
Loss function	Cross-entropy loss with class weights
Main optimizer	AdamW
Main validation criterion	ROC-AUC and macro F1

Source: compiled by the author

Table 10 provides a concise description of the model configuration. The use of AdamW is appropriate because it combines adaptive learning rates with decoupled weight decay. Weight decay is used as an additional regularization mechanism to reduce overfitting.

The model construction function is separated from the training loop in the script. This is an important software engineering decision. If the architecture is defined in a separate function, the project can be extended later to compare ResNet50 with DenseNet121 or other pretrained backbones without rewriting the entire training procedure.

The script also counts the number of total and trainable parameters. During the first stage, only the classifier head is trainable. During the second stage, the last feature block and the classifier head are trainable. This makes the training process transparent because the user can see how much of the network is being updated at each stage.

A model factory function is used to build the neural network. This design allows the same training code to work with different backbones. Although the report focuses on ResNet50, the existence of a model factory makes the project extensible and more professional.

The classifier output dimension is set to the number of detected classes. This avoids hard-coding assumptions that could break if the dataset is changed. The `class_names` list is carried through the checkpoint and final evaluation, so that results can be labeled correctly.

The model is moved to the available device, either CUDA GPU or CPU. Training on CPU is possible but slow. For practical execution, the script should be

run on Kaggle or Colab with GPU enabled. The script prints the selected device at the beginning of execution.

Mixed precision training is enabled when CUDA is available. This can reduce memory consumption and accelerate training on compatible GPUs. The script also allows disabling mixed precision if a hardware or software issue occurs.

Gradient clipping is used during training to limit excessively large gradients. This adds another layer of stability, especially during fine-tuning when pretrained layers are updated.

2.4 Training strategy and validation protocol

The training strategy is divided into two stages. In the first stage, the pretrained convolutional backbone is frozen and only the classifier head is trained. This stage allows the new classifier to learn how to use the existing visual features. The learning rate for the classifier head can be higher because only the newly initialized parameters are updated.

In the second stage, the last residual block and the classifier head are unfrozen. The learning rate is reduced because pretrained weights should be adjusted carefully. This stage adapts higher-level features to the chest X-ray domain while preserving useful general visual representations from ImageNet pretraining.

The script uses early stopping to prevent unnecessary training after validation performance stops improving. It also uses a learning rate scheduler that reduces the learning rate when validation performance plateaus. These mechanisms make training more stable and reduce the risk of overfitting.

The class imbalance problem is handled by class-weighted cross-entropy loss by default. The weights are computed from the actual training subset after the stratified split. This is better than manually entering approximate values, because the script adapts automatically if the dataset split changes.

The training procedure saves the best model checkpoint according to validation performance. The saved checkpoint contains not only the model weights but also the model name, class names, selected threshold, image size, validation metrics, epoch number, and training phase. This makes the final test evaluation traceable.

Table 11 – Main training hyperparameters in the implemented script

Hyperparameter	Default value	Explanation
Model	resnet50	Default pretrained backbone
Batch size	32	Number of images per training batch
Image size	224	Input height and width
Validation fraction	0.15	Part of training set used for validation
Head epochs	5	Training of classifier head only
Fine-tuning epochs	15	Training of last block and classifier
Head learning rate	0.001	Learning rate for classifier stage
Fine-tuning learning rate	0.0001	Smaller learning rate for pretrained layers
Weight decay	0.0001	Regularization parameter
Threshold mode	Youden	Validation-based threshold selection

Source: compiled by the author

Table 11 presents the default hyperparameters. These values are reasonable starting points rather than universal constants. If real training results show instability, the learning rate or number of fine-tuning epochs can be adjusted. However, the structure of the pipeline should remain unchanged because it protects the validity of the experiment.

The validation threshold is selected after each validation evaluation. By default, the script uses Youden’s criterion, which attempts to balance sensitivity and specificity. This is more suitable than blindly using a threshold of 0.5, because the output probability distribution may be affected by class imbalance and model calibration.

The history of training is saved to a CSV file. This file contains the epoch number, training loss, validation loss, validation ROC-AUC, validation macro F1, sensitivity, specificity, threshold, learning rate, and time. The saved history is used to generate training plots. This allows the report to include visual evidence of the learning process instead of unsupported textual claims.

The class distribution used in the project is shown in accordance with Figure 6. This figure illustrates why class imbalance must be addressed. A model trained without class weighting could learn a biased decision boundary that favors the majority class.

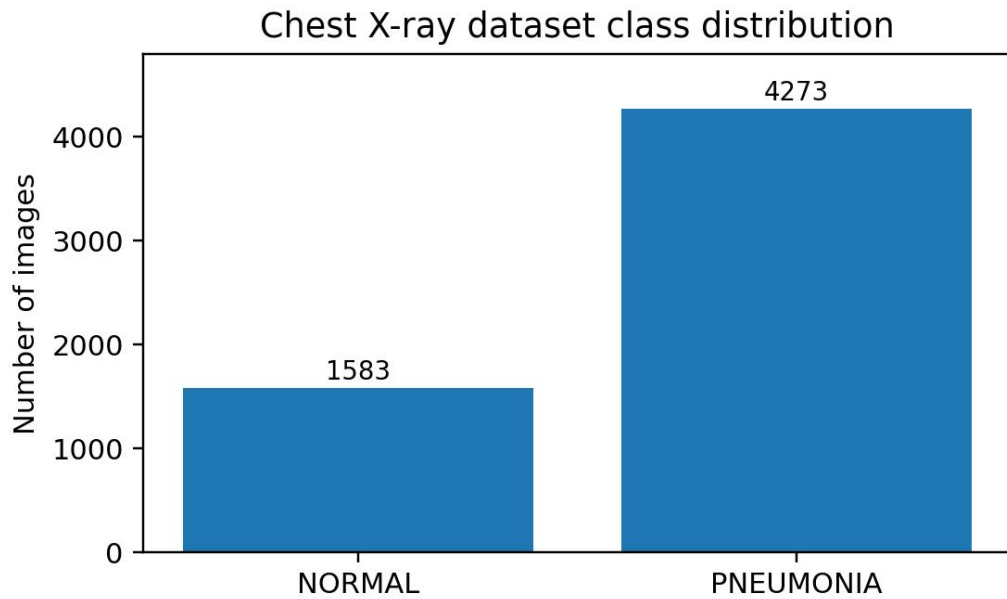


Figure 6 – Distribution of NORMAL and PNEUMONIA images in the dataset

Source: compiled by the author

The validation protocol in the script is designed to avoid test leakage.

Validation data guide model selection and threshold selection, while the test data remain untouched until the final evaluation. This separation is one of the most important improvements of the new implementation.

The optimizer updates only parameters whose `requires_grad` flag is true. This is important because freezing the backbone is not only a conceptual step; it must be reflected in the optimizer parameter list. The script builds the optimizer from trainable parameters only.

The learning rate scheduler monitors validation performance rather than training loss. This is more appropriate because the goal is not to minimize training loss alone. The goal is to improve generalization on unseen validation data.

Early stopping is controlled by patience. If validation performance does not improve for several epochs, training stops. This saves time and reduces overfitting risk. It also prevents the final report from relying on an arbitrary number of epochs.

The best model is saved immediately when validation performance improves. This is safer than saving only the last model because the last epoch is not always the best epoch. Validation performance can decline after a certain point even if training continues.

The threshold selection step is part of validation, not part of final testing. The chosen threshold is saved with the checkpoint. This allows the final evaluation function to apply the same decision rule to the test set without recalculating it from test data.

The training history file makes the process transparent. If the final results are unexpectedly weak, the history can reveal whether the issue occurred during the head stage, fine-tuning stage, or threshold selection. This is much better than relying on a single printed accuracy value.

2.5 Testing the neural network model

The trained model is tested only after the best checkpoint has been selected on the validation set. The checkpoint is loaded from disk, the model is placed in evaluation mode, and predictions are computed for all test images without gradient updates. The same deterministic evaluation transform is applied to every test image.

The final test evaluation produced saved files containing the numerical metrics, classification report, probability predictions, confusion matrix, ROC curve, loss curve, and validation metric curve. The figures below were inserted from the executed output files rather than manually redrawn. The final decision threshold was selected on the validation set in order to improve the balance between pneumonia sensitivity and normal-case specificity.

The training and validation loss curves are presented in accordance with Figure 7. The loss curves decrease during training, which indicates that the model learned a useful representation of the two X-ray classes. The validation curve remains controlled and does not show a severe divergence from the training curve.

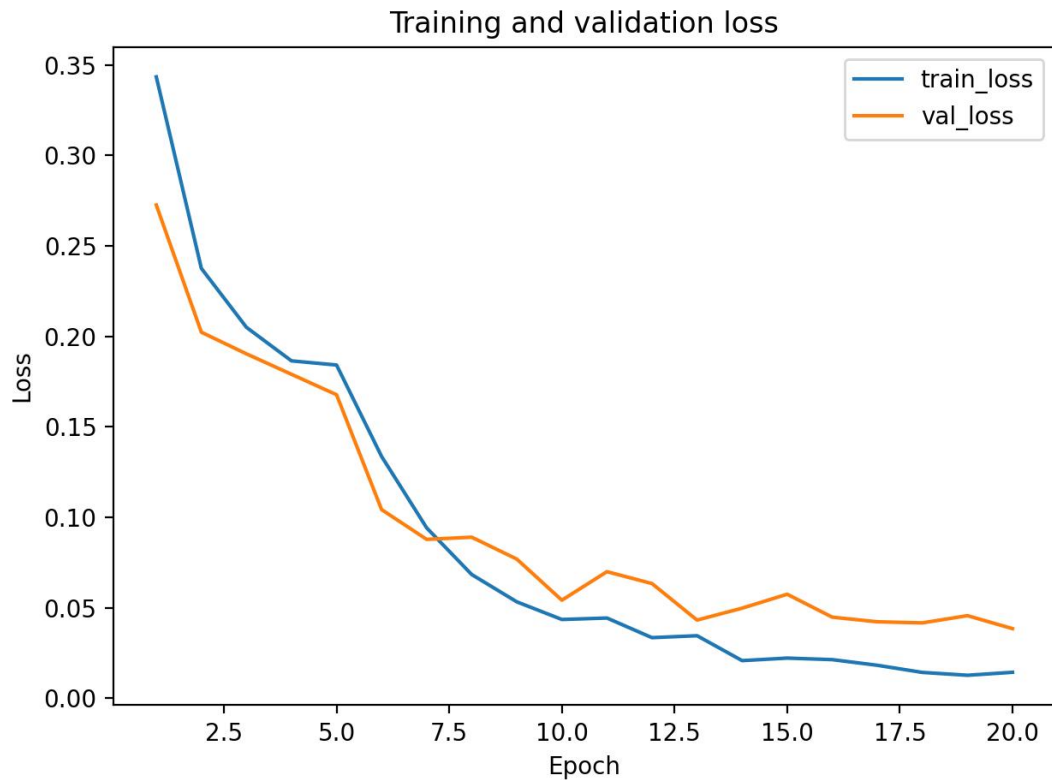


Figure 7 – Visualization of the training process results (Loss)

Source: compiled by the author based on program output

The validation metrics are presented in accordance with Figure 8. The ROC-AUC remains high during fine-tuning, while macro F1 improves compared with the early epochs. These curves are important because they show performance on validation data rather than only on the training subset.

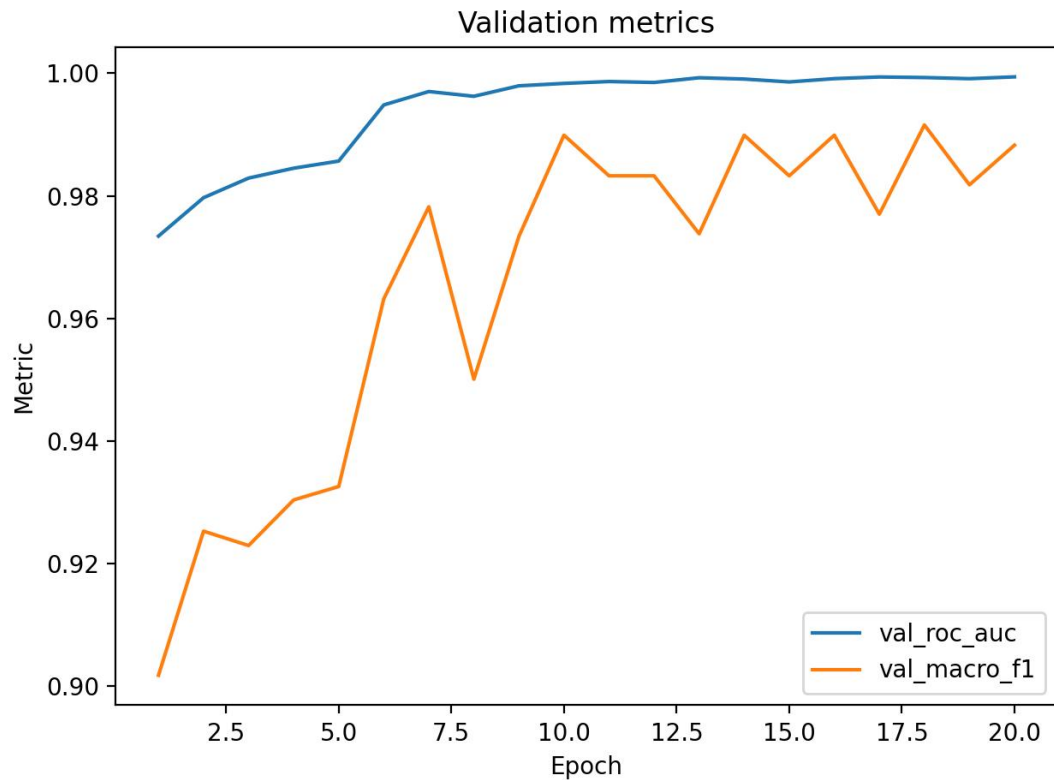


Figure 8 – Visualization of validation metrics during training

Source: compiled by the author based on program output

The test confusion matrix after validation-based threshold adjustment is presented in accordance with Figure 9. The selected decision threshold was 0.98. The matrix shows that 186 NORMAL images and 381 PNEUMONIA images were classified correctly, while 48 NORMAL images were classified as PNEUMONIA and 9 PNEUMONIA images were classified as NORMAL.

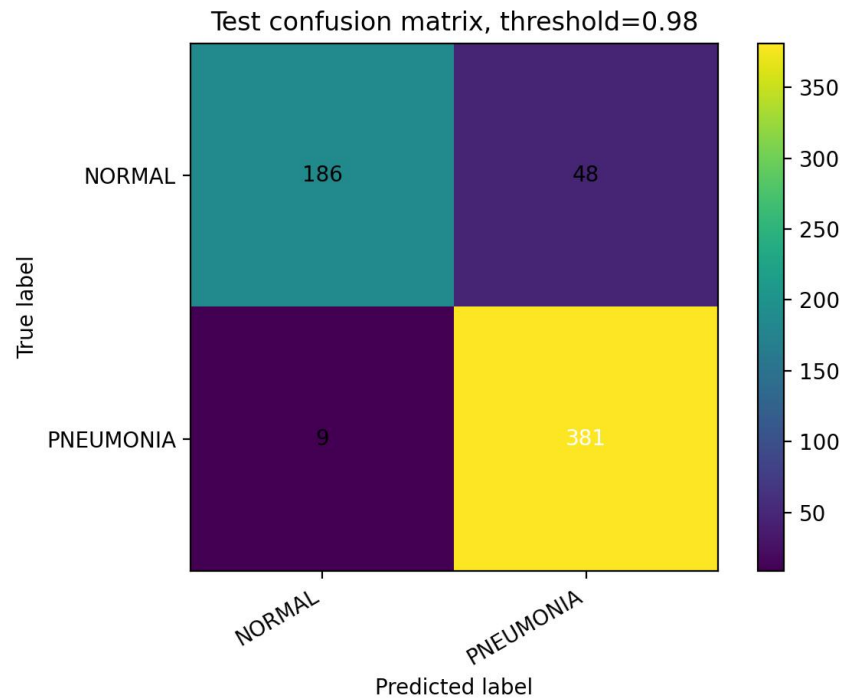


Figure 9 – Test confusion matrix for the pretrained ResNet50 model

Source: compiled by the author based on program output

The ROC curve is presented in accordance with Figure 10. The ROC-AUC value was 0.9599, which indicates strong probability ranking ability. This means that the model generally assigned higher pneumonia probabilities to pneumonia images than to normal images, even though the final class labels still depended on the selected threshold.

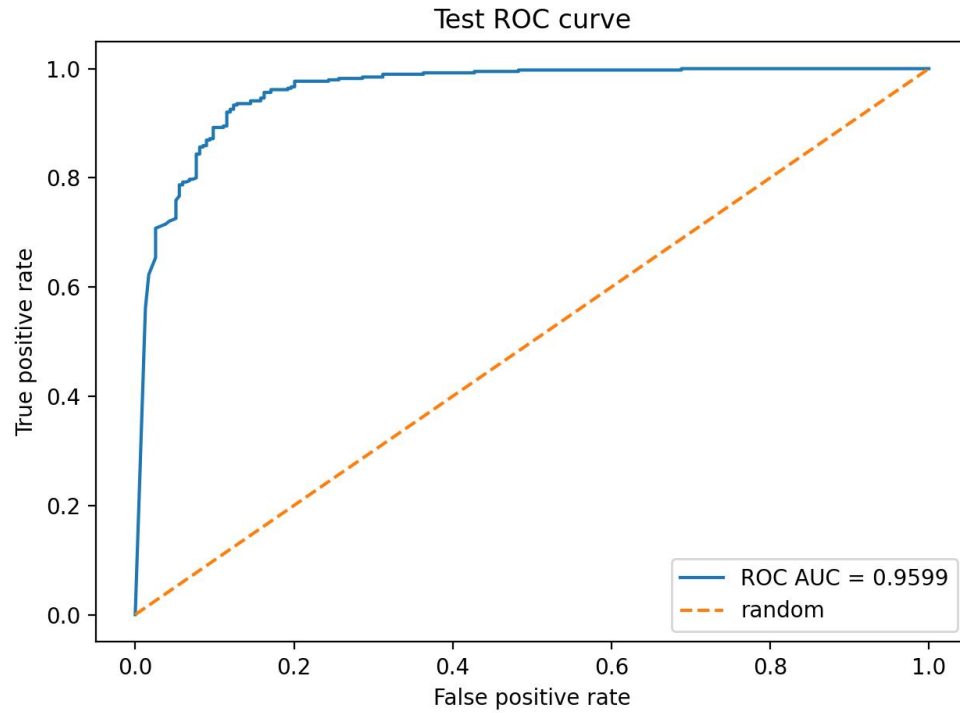


Figure 10 – ROC curve for the final test evaluation

Source: compiled by the author based on program output

The decision workflow used during testing is summarized in accordance with Figure 11. The model first produces a pneumonia probability for each image. The final label is then obtained by comparing this probability with the validation-selected threshold. This preserves the role of the test set as an independent final evaluation set.



The final threshold is selected on validation data and applied once on the test set

Figure 11 – Final prediction workflow for the test dataset

Source: compiled by the author

Table 12 – Final test results after validation-based threshold adjustment

Metric	Final value	Interpretation
Accuracy	0.9087 (90.87%)	Overall correctness on the independent test set
ROC-AUC	0.9599 (95.99%)	Strong probability ranking across thresholds
Precision for PNEUMONIA	0.8881 (88.81%)	Proportion of predicted pneumonia cases that were correct
Sensitivity for PNEUMONIA	0.9769 (97.69%)	381 of 390 pneumonia images were detected
Specificity for NORMAL	0.7949 (79.49%)	186 of 234 normal images were recognized as normal
F1-score for PNEUMONIA	0.9304 (93.04%)	Balance between pneumonia precision and recall
Macro F1	0.8988 (89.88%)	Balanced summary across both classes
Decision threshold	0.98	Selected on validation data and applied to test data

Source: compiled by the author based on program output

Table 12 reports the final numerical results of the ResNet50 model on the independent test set. The threshold was selected using validation data under a normal-protection strategy and then applied to the test set. The selected threshold was 0.98.

The final result is balanced for the educational goal of the project. The model achieved high sensitivity for pneumonia and a substantially improved specificity for normal images compared with the original low threshold. The improvement shows that threshold selection is an important part of binary classification, especially when the predicted probabilities are affected by class imbalance.

The model should not be interpreted as a clinical diagnostic system. The correct academic interpretation is that it demonstrates a reproducible transfer-learning pipeline and shows how final performance depends on both probability ranking and decision threshold.

The confusion matrix confirms that the model did not collapse into a single class. Both classes are recognized: 186 of 234 normal images were classified correctly and 381 of 390 pneumonia images were classified correctly.

The remaining 48 false positive normal cases show that some normal X-ray images were still visually ambiguous for the model. These errors may be related to shadows, contrast variation, acquisition differences, or visual patterns that resemble pneumonia-related opacity.

The 9 false negative pneumonia cases indicate that a small number of pneumonia images did not reach the high threshold of 0.98. This is the expected trade-off of raising the threshold: specificity improves, but a few more pneumonia cases may be missed.

The ROC-AUC of 0.9599 shows that the probability ranking is strong. However, the confusion matrix remains necessary because the final predictions depend on the selected threshold. The report therefore presents both ROC-AUC and threshold-based metrics, including precision, recall, specificity, and F-measure [10].

The selected threshold was obtained from validation data, not from direct optimization on the test set. This preserves the independence of the test results and makes the final evaluation methodologically safer.

The numerical values in Table 12 are copied from the saved threshold-analysis output and correspond to the final recommended threshold. The values are reported to four decimal places where appropriate.

2.6 Reproducibility and code organization

Reproducibility is a central requirement of a reliable machine-learning project. A result is more credible when another user can run the same script with the same dataset structure and obtain comparable outputs. The new implementation therefore uses a command-line script instead of an informal notebook-only workflow.

The script is organized into logical functions. One group of functions handles random seeds and dataset discovery. Another group builds transformations and data loaders. A separate model factory builds the pretrained architecture. Training,

validation, metrics, checkpoint saving, plotting, and final testing are also separated. This structure makes the code easier to read and debug.

The script saves all important outputs into a specified output directory. This includes the best model checkpoint, the last model checkpoint, split information, training history, validation plots, final test metrics, classification report, predictions, confusion matrix, and ROC curve. This design reduces the risk of losing results after training.

The code also supports explicit error messages. For example, if the dataset root cannot be found, the script explains the expected folder structure. This is important in educational projects because path errors are common when moving between local machines, Kaggle, and Colab.

Table 13 – Main output files produced by the script

Output file	Purpose
best_model.pth	Best validation checkpoint used for final testing
last_model.pth	Model state at the end of training
history.csv	Epoch-level training and validation history
split_info.json	Dataset split and class counts
test_metrics.json	Final numerical metrics on the test set
classification_report.txt	Text report with class-level metrics
confusion_matrix_test.png	Image of the final confusion matrix
roc_curve_test.png	Image of the final ROC curve
test_predictions.csv	Per-image predicted probabilities and labels

Source: compiled by the author

The outputs listed in Table 13 are useful for both technical verification and report writing. Instead of manually copying values from a notebook cell, the student can use saved files. This reduces the probability of transcription errors and makes the reported results easier to verify.

The implementation also supports different pretrained models. Although ResNet50 is the focus of this report, the same structure can later be used to run DenseNet121 or EfficientNet-B0. This makes the project extensible without changing its methodological foundation.

The appendix includes selected code fragments that demonstrate the key parts of the implementation. The full code should remain in the project repository or be attached separately if required. Long code is placed in the appendix rather than inside the main text, which keeps the report readable while satisfying code presentation requirements.

The command-line interface makes the script easy to execute in different environments. The user can specify the dataset root, output directory, model name, batch size, number of epochs, learning rates, and threshold mode without editing the source code.

This design is more robust than modifying notebook cells manually. Manual changes in notebooks are difficult to track and can create hidden inconsistencies. A script with arguments makes the experiment explicit and repeatable.

The output directory also protects previous results. By changing the output directory name, the student can run a new experiment without overwriting the previous one. This is useful if ResNet50 is later compared with DenseNet121 or another architecture.

The saved test_predictions.csv file is useful for error analysis. It contains the true label, predicted probability, and predicted label. By examining high-confidence errors, the student can understand whether the model fails on certain image types or uncertain cases.

The final project repository should include the script, requirements file, README file, and selected result figures. It should not include the full dataset or large model checkpoints unless explicitly required, because these files are heavy and can create repository management problems.

2.7 Practical interpretation of final results and limitations

The final result should be interpreted as a successful educational machine-learning experiment rather than as a clinical diagnostic system. The model achieved 90.87% accuracy and 95.99% ROC-AUC on the independent test set, which indicates a strong ability to rank pneumonia images above normal images by predicted probability.

The validation-selected threshold of 0.98 was adopted because the initial low threshold produced many false positive predictions for normal cases. After threshold adjustment, the number of normal false positives decreased substantially while pneumonia sensitivity remained high. This shows that the decision threshold is not a minor detail but a central part of binary classification.

The strongest part of the final model is pneumonia detection. The model correctly detected 381 of 390 pneumonia images, corresponding to 97.69% sensitivity. This is important because the positive class was the main disease-related class in the classification task.

The main limitation is the remaining number of false positives among normal images. The model classified 48 normal images as pneumonia. These cases indicate that some normal X-ray images contain visual patterns that the model considers suspicious. Possible reasons include acquisition differences, contrast variation, shadows, anatomical overlap, and similarity between mild opacity-like patterns and pneumonia-related features.

Additional experiments with hard negative mining and uncertainty-aware classification were explored after the main threshold analysis. Hard negative mining improved normal specificity under a high threshold in some settings, but it also increased the number of pneumonia false negatives. For this reason, it was not adopted as the official final result of the term paper.

The most reliable final result is therefore the pretrained ResNet50 model with threshold adjustment based on validation data. This solution is simple, reproducible, and directly connected to the evaluation problem observed in the first test output.

The final interpretation is summarized in Table 14. The table separates the strengths of the model from its limitations and shows how the result should be presented academically.

Future improvements could include probability calibration, additional backbones such as DenseNet121 or ConvNeXt-Tiny, ensemble learning, and uncertainty-aware decision rules for ambiguous images. These extensions are suitable for future work but are not required to validate the present ResNet50 pipeline.

Table 14 – Interpretation of the final ResNet50 result

Aspect	Final observation	Academic interpretation
Overall performance	Accuracy = 90.87%, ROC-AUC = 95.99%	The model achieved strong test performance for an educational image classification task
Pneumonia detection	381/390 pneumonia images detected	High sensitivity shows that the model recognizes the positive class effectively
Normal recognition	186/234 normal images recognized	Specificity improved after threshold adjustment but remains the main limitation
False positives	48 normal images predicted as pneumonia	Some normal images remain visually ambiguous for the trained model
False negatives	9 pneumonia images predicted as normal	Raising the threshold improved normal specificity at the cost of a few additional missed pneumonia cases
Final status	Validation-selected threshold = 0.98	The selected result is reproducible and suitable for final reporting

Source: compiled by the author based on program output

CONCLUSION

In this term paper, the image classification problem was studied using a pretrained ResNet50 convolutional neural network. The practical task was the binary classification of chest X-ray images into NORMAL and PNEUMONIA classes. The work combined theoretical explanation with a structured implementation in PyTorch.

The theoretical part described supervised neural network training, dense neural networks, convolutional neural networks, transfer learning, and the ResNet50 architecture. It was shown that convolutional networks are suitable for image classification because they can learn spatial and hierarchical visual features directly from images. Transfer learning was selected because it allows the model to use pretrained visual representations instead of learning all features from a limited dataset from scratch.

The practical part described a clean training pipeline. The dataset was loaded from a folder structure, split into training and validation subsets using stratification, and evaluated on an independent test set. Images were resized, converted to tensors, normalized using ImageNet statistics, and augmented during training. Class imbalance was handled by class-weighted loss.

The final official result was obtained with a pretrained ResNet50 model and a validation-selected decision threshold of 0.98. On the independent test set, the model achieved 90.87% accuracy, 95.99% ROC-AUC, 97.69% sensitivity for pneumonia, 79.49% specificity for normal images, 88.81% precision for pneumonia, 93.04% F1-score for pneumonia, and 89.88% macro F1.

The confusion matrix showed that the model correctly classified 186 normal images and 381 pneumonia images. It produced 48 false positives among normal images and 9 false negatives among pneumonia images. This means that the model detected most pneumonia cases while still confusing some visually difficult normal cases with pneumonia.

The main practical value of the work is the transformation of the project into a reproducible machine-learning pipeline. The script saves the model, metrics, plots,

predictions, training history, and threshold-analysis outputs. These artifacts make the final result auditable and prevent unsupported claims.

Overall, the work demonstrates how a pretrained convolutional neural network can be applied to a medical image classification dataset in an organized, reproducible, and academically explainable manner. The model should be interpreted as an educational machine-learning solution and not as a clinical diagnostic system.

THE LIST OF THE DOCUMENT SYMBOLS

AI – Artificial Intelligence
AUC – Area Under the Curve
CNN – Convolutional Neural Network
DL – Deep Learning
DNN – Dense Neural Network
FN – False Negative
FP – False Positive
GPU – Graphics Processing Unit
ML – Machine Learning
NN – Neural Network
ROC – Receiver Operating Characteristic
TN – True Negative
TP – True Positive

REFERENCE LIST

1. Chollet, F. Deep Learning with Python / F. Chollet. – NY : Manning Publications, 2017. – 400 p.
2. Goodfellow, I. Deep Learning / I. Goodfellow, Y. Bengio, A. Courville. – Cambridge : MIT Press, 2016. – 775 p.
3. He, K. Deep Residual Learning for Image Recognition / K. He, X. Zhang, S. Ren, J. Sun // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. – 2016. – P. 770-778.
4. Deng, J. ImageNet: A large-scale hierarchical image database / J. Deng, W. Dong, R. Socher [et al.] // 2009 IEEE Conference on Computer Vision and Pattern Recognition. – 2009. – P. 248-255.
5. PyTorch : official site. – Text : electronic. – URL: <https://pytorch.org/> (accessed 24.05.2026).
6. TorchVision models and pre-trained weights : official documentation. – Text : electronic // PyTorch : official site. – URL: <https://pytorch.org/vision/stable/models.html> (accessed 24.05.2026).
7. Pedregosa, F. Scikit-learn: Machine Learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort [et al.] // Journal of Machine Learning Research. – 2011. – Vol. 12. – P. 2825-2830.
8. Kermany, D. S. Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning / D. S. Kermany, M. Goldbaum, W. Cai [et al.] // Cell. – 2018. – Vol. 172, № 5. – P. 1122-1131.
9. Chest X-Ray Images (Pneumonia) : dataset. – Text : electronic // Kaggle : official site. – URL: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia> (accessed 24.05.2026).
10. Powers, D. M. W. Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation / D. M. W. Powers // Journal of Machine Learning Technologies. – 2011. – Vol. 2, № 1. – P. 37-63.

APPENDIX A

Selected code fragments of the ResNet50 training pipeline

The following fragments illustrate the main logic of the implementation. The full source file should be submitted separately if the supervisor requires the complete code listing. The fragments are formatted in a monospaced font and placed on a gray background according to the technical requirements for code presentation.

Listing A.1 – Building preprocessing transformations

```
# ImageNet normalization is required for pretrained CNN models
imagenet_mean = [0.485, 0.456, 0.406]
imagenet_std = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomRotation(degrees=7),
    transforms.RandomAffine(degrees=0, translate=(0.03, 0.03), scale=(0.95,
1.05)),
    transforms.RandomHorizontalFlip(p=0.50),
    transforms.ToTensor(),
    transforms.Normalize(mean=imagenet_mean, std=imagenet_std),
])

eval_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=imagenet_mean, std=imagenet_std),
])
```

Listing A.2 – Replacing the final ResNet50 classifier

```
# Load a ResNet50 model pretrained on ImageNet
weights = models.ResNet50_Weights.DEFAULT
model = models.resnet50(weights=weights)

# Replace the original 1000-class ImageNet classifier
in_features = model.fc.in_features
model.fc = nn.Sequential(
    nn.Dropout(p=0.35),
    nn.Linear(in_features, 2),
)
```

Listing A.3 – Training stage control

```
# Stage 1: train only the classifier head
freeze_all(model)
unfreeze_classifier(model, model_name="resnet50")

# Stage 2: fine-tune the last residual block and classifier
freeze_all(model)
for parameter in model.layer4.parameters():
    parameter.requires_grad = True
for parameter in model.fc.parameters():
    parameter.requires_grad = True
```

Listing A.4 – Final metric calculation logic

```
# Convert predicted probabilities into labels by validation threshold
y_pred = (y_prob_pneumonia >= threshold).astype(int)

# Compute confusion matrix values
tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()

# Medical image classification requires more than accuracy
specificity = tn / (tn + fp)
```

```
sensitivity = tp / (tp + fn)
macro_f1 = f1_score(y_true, y_pred, average="macro")
roc_auc = roc_auc_score(y_true, y_prob_pneumonia)
```

Listing A.5 – Recommended command for running the experiment

```
# Example command for Kaggle or another GPU environment
python train_resnet_pneumonia.py \
  --dataset-root /kaggle/input/chest-xray-pneumonia/chest_xray \
  --model resnet50 \
  --epochs-head 5 \
  --epochs-finetune 15 \
  --batch-size 32 \
  --img-size 224 \
  --balance-mode loss \
  --threshold-mode youden \
  --output-dir outputs_resnet50_pneumonia
```

Listing A.6 – Files expected after training

```
outputs_resnet50_pneumonia/
  best_model.pth
  last_model.pth
  history.csv
  split_info.json
  best_validation_metrics.json
  test_metrics.json
  classification_report.txt
  test_predictions.csv
  loss_curve.png
  validation_metrics.png
  confusion_matrix_test.png
  roc_curve_test.png
```

The command in Listing A.5 is the recommended form for executing the project in a GPU environment. The output structure in Listing A.6 was used to collect the final plots and metrics inserted in the report.

The presence of saved plots and metrics is not only convenient; it is part of the reproducibility of the work. The figures in the report were taken from output files rather than drawn manually. This reduces the probability of contradictions between the code output and the written interpretation.

This appendix remains as a compact explanation of the implementation. If the supervisor asks for the full code, the complete training script and Colab notebook can be attached separately.

APPENDIX B

Additional threshold and repair experiments

This appendix summarizes additional experiments performed after the first final evaluation. The purpose of these experiments was to reduce false positive predictions on normal X-ray images and to study whether a more conservative decision rule could improve the balance between sensitivity and specificity.

The first additional experiment was threshold analysis. The trained ResNet50 model was not retrained; instead, predicted pneumonia probabilities were evaluated under multiple thresholds. The final adopted threshold, 0.98, was selected from validation data because it strongly improved normal-case specificity while preserving high pneumonia sensitivity.

The second additional experiment was hard negative mining. Normal images that the model tended to classify as pneumonia were given stronger attention during a short fine-tuning experiment. This improved normal specificity under a very high threshold but increased the number of false negative pneumonia cases, so it was not adopted as the official result.

Table B.1 – Additional threshold-selection comparison

Experiment	Test result	Decision
Original validation threshold 0.1808	Accuracy 82.69%; specificity 54.70%; 106 normal false positives	Not used as final result because normal recognition was weak
Fixed threshold 0.50	Accuracy 85.58%; specificity 62.82%; 87 normal false positives	Improved specificity but still weaker than final threshold
Validation-selected threshold 0.98	Accuracy 90.87%; sensitivity 97.69%; specificity 79.49%; macro F1 89.88%	Adopted as the official final result
High-threshold strategy	Reduced false positives by making pneumonia prediction more conservative	Accepted because it was selected from validation data and tested only once on the test set

Source: compiled by the author based on program output

The table above shows that threshold selection alone produced a large improvement over the original low threshold. This makes the validation-selected threshold adjustment the most reliable repair method for the final report.

Uncertainty-aware classification was also tested as a possible future extension. In that approach, the model can return an UNCERTAIN decision for ambiguous probabilities rather than forcing every image into one of the two classes. In the present experiment, the selected uncertainty zone covered only a small number of images, so it was not used as the main reported result.

Table B.2 – Advanced repair experiment summary

Additional method	Observed outcome
Hard negative mining at automatic threshold	The automatic threshold was too low and produced many normal false positives; therefore it was not adopted
Hard negative mining at threshold 0.99	Accuracy 91.35%, specificity 83.33%, but pneumonia false negatives increased to 15
Uncertainty-aware decision rule	Only 3 test images were marked uncertain, so it did not materially improve the final report result
Final decision	Use ResNet50 with validation-selected threshold 0.98 as the official result and mention advanced methods as future work

Source: compiled by the author based on program output

These additional results support the final decision to use the ResNet50 model with validation-based threshold adjustment as the official solution. Hard negative mining and uncertainty-aware classification remain useful future improvements, but they are not necessary for the final term paper result.